As a Senior API Architect, I have designed the REST API for the Banking Management System, prioritizing security, auditability, and clear resource segregation.

---

1. API Overview

- **Base URL:** `https://api.bankingsystem.com/v1`
- **Protocol:** HTTPS only
- **Data Format:** JSON
- **Versioning:** URI-based versioning (`/v1/...`)
- **Timezone:** All timestamps are strictly in ISO 8601 (UTC).

2. Authentication

- **Scheme:** OAuth 2.0 + OpenID Connect (OIDC).
- **Flow:** Authorization Code Flow with PKCE for mobile/web.
- **MFA:** After the primary token is issued, if a sensitive operation (e.g., transfer > $1k) is triggered, the API requires an `X-MFA-Token` header containing a short-lived TOTP validation code.
- **Security Header:** `Authorization: Bearer `

---

3. Key Endpoints

| Method | Endpoint | Description | Role |

| :--- | :--- | :--- | :--- |

| POST | `/auth/mfa/verify` | Validate TOTP code | Any |

| GET | `/accounts` | List user accounts | Customer |

| POST | `/transactions` | Initiate a transfer | Customer |

| GET | `/loans/pending` | View loan queue | Employee |

| PATCH | `/loans/{id}` | Approve/Reject loan | Employee |

| GET | `/admin/fraud-alerts` | View flagged transactions | Admin |

---

4. Sample Request/Response Bodies

A. Execute Transaction

POST `/transactions`

```
{
  "from_account_id": 101,
  "to_account_id": 202,
  "amount": 15000.00,
  "currency": "USD"
}
```

Response (202 Accepted)

```
{
  "transaction_id": 9982,
  "status": "FLAGGED_FOR_REVIEW",
  "message": "Transaction exceeds limit. Pending fraud verification."
}
```

---

5. Status Codes

- **200 OK:** Request successful.
- **201 Created:** Resource created (e.g., new loan application).
- **202 Accepted:** Request received, processing (e.g., asynchronous fraud checks).
- **400 Bad Request:** Validation failure.
- **401 Unauthorized:** Missing or expired token.
- **403 Forbidden:** Insufficient permissions (e.g., Customer accessing Admin endpoint).
- **429 Too Many Requests:** Rate limiting.
- **500 Internal Server Error:** Unexpected server-side fault.

---

6. Validation & Error Responses

We use a standardized error format to facilitate consistent frontend handling.

Validation Logic:

- **Amount:** Must be > 0.

- **Accounts:** Sender must own the `from_account`.
- **MFA:** Required for any transaction exceeding daily account limits.

Standard Error Body:

```
{
  "error": {
    "code": "INSUFFICIENT_FUNDS",
    "message": "The account balance is insufficient for this transaction.",
    "request_id": "req-xyz-123",
    "details": {
      "current_balance": 500.00,
      "requested_amount": 1500.00
    }
  }
}
```

---

7. Architectural Best Practices Implemented

1. **Idempotency:** All `POST /transactions` requests include an `Idempotency-Key` header (UUID) to prevent double-charging on network retries.

2. **Audit Logs:** Middleware intercepts all authenticated requests and logs them to the `Audit_Logs` table asynchronously via a message queue (e.g., RabbitMQ or Kafka) to ensure zero latency impact on the user.

3. **HATEOAS:** API responses include links to related actions (e.g., a loan response will include a `links` object for `approve` or `reject` actions).

4. **Rate Limiting:** Implemented via Redis, limiting customers to 100 requests per minute and higher tiers for internal system accounts.

5. **Security:** All PII and financial sensitive fields are masked in logs. Database TDE and TLS 1.3 are strictly enforced.